# GVCCTurbo: Multi-Resolution Coding and Predictive Skip for Generative Video Compression

Anonymous WACV Algorithms Track submission

Paper ID *****

## Abstract

Generative video codecs that drive a pretrained rectified-flow generator with a transmitted noise codebook recently achieved competitive perceptual quality at ultra-low bitrates, but their decode-time cost—tens to hundreds of generator forward passes per Group of Pictures (GOP)—is one to two orders of magnitude above the operating point of practical learned video codecs. We present a turbo-charged variant of this design (GVCCTurbo) that retains the underlying zero-shot codebook scheme but accelerates per-GOP decoding by up to 1.9× on the published Wan 2.1 baselines, with quality essentially preserved at a small bitrate increase.

Two complementary mechanisms drive the speedup. First, a multi-resolution coding schedule runs the early high-noise iterations at a fraction of the full spatial resolution and only switches to the target resolution near the clean end. We identify and fix a previously overlooked target-domain mismatch between low- and high-resolution stages: encoding the low stage on the spatially downsampled video is not equivalent to taking the DCT-low projection of the high-resolution latent, and naively chaining the two imposes a structural ceiling of approximately 1.4 dB. Unifying both stages in the high-resolution latent domain via DCT projection collapses this gap to 0.2 dB with only a small bitrate change. Second, a predictive-skip mechanism caches the generator's clean-latent estimate rather than its velocity, allowing every other generator call along the trajectory to be reconstructed from the cached endpoint. Combined with a one-step warmup after each resolution transition, this yields an overall 1.7–1.9× decoder speedup with a small quality cost ($<$ 0.3 dB PSNR in our main setting). Both fixes are obtained from a trajectory- geometric reading of the generator's flow, with negligible engineering overhead.

We evaluate GVCCTurbo on UVG primarily in the T2V and FLF2V GOP- conditioning modes; the I2V mode is included for completeness through its published configuration. Our main full-UVG result is at 720p in T2V mode: the turbo pipeline reduces per-GOP decoder wall-clock to 19 s ($\approx 1.8\times$ over the same pipeline without our two mechanisms) with PSNR within 0.3 dB of single-resolution coding at the same bitrate. As a preliminary 1080p result, on the first half of each UVG sequence in first-last-frame conditioning, the turbo pipeline matches the published GVCC operating point in bitrate and exceeds it by +0.5 dB PSNR and $-14\%$ LPIPS at approximately $2\times$ the decode speed. Code, per-GOP measurements, and codebook bitstreams will be released to support reproducibility.

## 1. Introduction

Practical video compression has long been dominated by spatial-temporal hybrid codecs in the HEVC/VVC tradition, in which intra-coded keyframes anchor a sequence of motion-compensated predictions and small per-block residuals are entropy-coded. Learned video codecs of the DCVC family [5, 7] have closed and in many cases overtaken the rate–distortion (RD) frontier of these traditional schemes while preserving the same Group-of-Pictures (GOP) intuition: a small, strongly conditioned model produces a deterministic reconstruction, and the transmitted bits describe how much the actual frames differ from that reconstruction.

A complementary line of work has recently shown that pretrained generative video models can themselves serve as the decoder of an ultra-low-bitrate codec. In this setting, the encoder transmits not a residual against a deterministic prediction, but a compact specification of which sample of the model's stochastic generation process to reproduce. GVCC [23] instantiates this idea by converting a rectified-flow video generator into a marginal-preserving stochastic process, and

replacing the per-step Gaussian noise of that process with codebook- indexed innovations that the encoder selects to match the ground-truth video. The result is a zero-shot codec—no training of the generator or any auxiliary network—that on UVG achieves substantially lower LPIPS than HEVC, VVC, and even strong learned codecs at sub-0.01 bpp.

The clear weakness of this design is decoder time. Each GOP requires $T \approx 20$ forward passes through a multi-billion-parameter video generator, plus a per-step codebook search; on the published Wan 2.1-14B [20] backbone, a single 33-frame 1080p GOP takes several hundred seconds to decode on a high-end GPU. This is one to two orders of magnitude above the operating point of DCVC-RT and rules out the codec in any latency-sensitive scenario.

In this paper we present GVCCTurbo, a turbo-charged variant of the same generative-codec design. The central design constraint is trajectory preservation: the decoder should still traverse the same fine-grained SDE/codebook correction process, because those per-step innovations are the transmitted residual channel of the codec. We therefore do not simply reduce the number of solver steps. Instead, we reduce the cost of some steps while keeping the codebook trajectory reproducible. Our two contributions are simple, trajectory-geometric, and require no retraining.

Predictive skip via clean-endpoint caching. The dominant per-step cost is the video generator's velocity evaluation. We replace roughly half of these evaluations along the SDE trajectory with a cached reconstruction. The naive way to cache—storing the previous step's velocity and reusing it—silently drifts, because the velocity is a function of both state and time. Instead, we cache the generator's clean-latent estimate (the trajectory endpoint) and reconstruct the skipped step's velocity from the current state and current time. Under the linear flow interpolant, the clean endpoint is invariant along the trajectory, so this scheme is geometrically faithful within a local linearization. We show empirically that the cache has an effective radius of one step: a fixed alternating skip/run schedule, combined with a one-step warmup after each resolution transition, is Pareto-optimal under this caching scheme; more aggressive multi-step skipping is unstable.

Multi-resolution coding with a unified target. A natural way to cut decoder cost is to run the early high-noise iterations of the generator at a lower spatial resolution and only switch to the target resolution near the clean end—a video-coding analogue of progressive spectral diffusion [21]. We find that the obvious instantiation (encode at low-res VAE, switch to high-res VAE via a spectral lift) hits a structural ceiling of roughly 1.4 dB on UVG that persists across backbone scale and stage count, and that does not close under a joint oracle. The underlying cause is that the low-resolution VAE latent and the DCT-low projection of the high-resolution VAE latent are different objects, not aligned by the spectral lift used at transition. Unifying both stages in the high-resolution latent domain—using the DCT-low projection of the high-res latent as the low-stage codebook target—collapses the ceiling to 0.2 dB at the same bitrate, recovering nearly all of the gap with a one- line change in the codec target.

Negative results as design constraints. Several plausible accelerations fail for principled reasons. Band-limiting the codebook innovation breaks the full-spectrum noise distribution expected by the SDE; caching velocity remembers the previous direction rather than the current destination; multi-step cache reuse compounds its own extrapolation error; and a naive low-resolution VAE target moves the codec into the wrong latent domain. These failures shape the final design: GVCCTurbo reduces generator cost only when the missing computation can be replaced without changing the stochastic correction channel that carries the compressed bits.

GOP-level evaluation across conditioning modes. GVCC is evaluated in three GOP-conditioning modes: text-only (T2V), with a single anchor frame (I2V) with autoregressive chaining, and with both endpoint frames (FLF2V) with boundary sharing across consecutive GOPs. We apply our acceleration mechanisms primarily to T2V and FLF2V; the I2V mode is included by reusing the published GVCC-I2V configuration without re-evaluation. Our main full-UVG measurement is at 720p in T2V mode: the full acceleration stack reduces per-GOP decoder wall-clock to 19 s with PSNR within 0.3 dB of the single-resolution baseline at the same bitrate. At 1080p we report preliminary half-UVG results (first 9 GOPs per sequence): in FLF2V mode the turbo pipeline matches the bitrate of the published GVCC-FLF2V and exceeds it by +0.5 dB PSNR and $-14\%$ LPIPS at approximately $2\times$ the decode speed, and in T2V mode (Wan-T2V-1.3B) it reaches 0.0030 BPP at 108 s per GOP. Full-UVG 1080p measurements are the next step.

The remainder of the paper is organized as follows. Section 2 positions our work against traditional, learned, and generative video codecs. Section 3 describes the pipeline and derives both acceleration mech-

anisms, including the trajectory-geometric arguments and falsified alternatives. Section 4 reports the main RD and decode-time comparison on UVG. Section 5 isolates the contribution of each component and validates the one-step-radius result for predictive skipping.

## 2. Related Work

Traditional and learned video codecs. Standardized hybrid codecs HEVC [18] and VVC [2] structure a sequence into intra (I), predicted (P), and bidirectional (B) frames, transmitting residuals against motion-compensated predictions through DCT and entropy coding. Learned video codecs largely retain this GOP layout but replace each component with neural modules: DCVC and its descendants [5, 7] use learned conditional entropy models and feature-space prediction. These methods now match or exceed VVC in PSNR at the standard 0.05–0.20 bpp operating range with real-time-class decode speeds, but their RD curves rapidly flatten below 0.01 bpp.

Generative video compression. A separate line of work uses generative models to sample a plausible reconstruction at ultra-low bitrates rather than regress to a deterministic prediction. GLC-Video [15] and GNVC-VD [12] train end-to-end generative codecs that achieve 0.10–0.15 LPIPS at 0.003 bpp, well below the LPIPS of traditional and learned codecs at the same rate, but require dedicated training. Free-GVC [9] avoids training by performing reverse channel coding along the trajectory of a pretrained CogVideoX generator. GVCC [23] is the closest predecessor to our work: it converts a pretrained rectified-flow video generator into a stochastic process and transmits codebook-indexed per-step noise. We inherit GVCC's codec backbone—including its Turbo-DDCM [19] multi-atom codebook construction—and focus exclusively on closing the decode-time gap left open by that work.

Multi-resolution diffusion sampling. The idea of running the early high-noise iterations of a diffusion or flow model at lower resolution is well established in the sampling literature. Spectral Progressive Diffusion (SPD) [21] provides a clean spectral-domain reading: at each timestep $t$, only those frequency bands whose signal-to-noise ratio crosses a threshold need to be active, and lower bands cross first. Cascaded diffusion [4] and several text-to-image systems [3, 14] exploit the same structure with multiple separately trained models.

Our use of multi-resolution coding differs from these in two ways. First, the goal is not to generate plausible samples but to drive an encoder- specified target with bit-exact reproducibility on both sides. Second, the low and high stages share a single pretrained backbone (no cascading of distinct models), which forces a precise specification of what the low stage's target latent is—a question that does not arise in generation, where any plausible low-frequency content is acceptable. The target-domain mismatch we identify in Section 3.3 arises exactly because the low stage is anchored by the encoder, not free to drift.

Accelerating diffusion sampling. Reducing the number of model evaluations is well studied in the distillation literature: progressive distillation [16], consistency models [10, 17], and Sana [22]-style mobile distillations all aim to bring sample counts from $T = 20$–$50$ down to $T \leq 4$. These approaches require retraining and produce a distilled generator that may not be compatible with the encoder's per-step codebook reproducibility requirement. Closer to us are inference-time caching schemes that reuse intermediate quantities across denoising steps: DeepCache [11] caches internal U-Net features, and several recent works [6] cache velocity outputs directly. We adopt the latter family but cache the clean-latent endpoint rather than the velocity (Section 3.4); this choice is forced by the codec requirement that the cache stay valid across an SDE step where the trajectory state changes by an encoder-selected codebook innovation.

Pretrained video generators as the decoder backbone. We instantiate GVCCTurbo on Wan 2.1 [20], a rectified-flow video generator supporting unconditional (T2V), single-anchor (I2V), and dual-anchor (FLF2V) generation modes at 480p and 720p+. The choice follows [23] for direct comparability; the framework inherits whatever resolution and conditioning support the base generator provides.

## 3. Method

We treat each $F$-frame GOP as the unit of compression. The decoder reconstructs the GOP by running a pretrained rectified-flow video generator forward for $T$ discrete steps; the encoder selects per-step codebook indices so that the resulting trajectory matches the ground-truth GOP. Section 3.1 states this codec backbone in codec-centric terms (inherited from [19, 23] in compact form), and the remaining subsections describe the two acceleration mechanisms.

### 3.1. Codec backbone

Encoder/decoder pipeline. A pretrained 3D VAE maps a GOP $\mathbf{V} \in \mathbb{R}^{F\times H\times W\times 3}$ to a latent $\mathbf{x}_0 \in \mathbb{R}^{F_l\times H_l\times W_l\times C}$. The decoder is a rectified-flow video generator whose ODE $\dot{\mathbf{x}}_t = \mathbf{u}_\theta(\mathbf{x}_t, t)$ transports samples from $t=1$ (pure noise) to $t=0$ (clean latent). We discretize this trajectory at $T$ timesteps $\{t_k\}_{k=1}^{T}$ and inject stochastic innovations at the first $T_{\text{sde}}$ of them, leaving an $N=T-T_{\text{sde}}$-step deterministic ODE tail:

$$\mathbf{x}_{t_{k+1}} = \mathbf{x}_{t_k} - f_{t_k}\Delta t_k + g_{t_k}\sqrt{\Delta t_k}\,\mathbf{z}_k^*, \tag{1}$$

where $f_{t_k}, g_{t_k}$ are the SDE drift and diffusion coefficients derived from $\mathbf{u}_\theta$ as in [23], and $\mathbf{z}_k^*$ is the per-step innovation that the encoder transmits via a $\log_2(K)$-bit codebook index together with a sign bit, per latent frame. The full bitstream therefore consists of $T_{\text{sde}}\cdot F_l\cdot M\cdot(\log_2 K+1)$ codebook bits per GOP, plus any boundary-frame bits required by the conditioning mode (Section 3.5).

Codebook construction. We use the multi-atom construction of Turbo-DDCM [19]: at step $k$ and latent frame $f$, the encoder generates $K$ i.i.d. Gaussian atoms with a deterministic seed shared by both sides, computes inner products with the per-frame residual $\mathbf{r}_{k,f} = \mathbf{x}_{0,f} - \hat{\mathbf{x}}_{0|t_k,f}$, and transmits the $M$ atoms with the largest absolute inner product together with their signs. The decoder regenerates the same $K$ atoms, indexes into them, and reconstructs $\mathbf{z}_{k,f}^*$ as a unit-variance sum.

Cost. A single decode requires $T$ video-generator evaluations and $T_{\text{sde}}$ codebook reconstructions. At $T=20$, $N=3$, $K=16384$, $M=80$, the per-GOP decoder cost on a 14B-parameter Wan 2.1 model at 1080p is approximately 700 s on an RTX PRO 6000. The two acceleration mechanisms below target the dominant cost—the per-step generator evaluation—without changing the codec backbone.

### 3.2. Trajectory-preserving acceleration principle

The key distinction from ordinary fast sampling is that GVCC is a codec: the bitstream specifies the stochastic innovations $\{\mathbf{z}_k^*\}$, and the decoder must reproduce the same trajectory as the encoder. Directly reducing $T_{\text{sde}}$ therefore removes both solver steps and opportunities to inject codebook residuals. GVCCTurbo instead keeps the correction grid

$$\mathcal{K}_{\text{sde}} = \{1,\dots,T_{\text{sde}}\} \tag{2}$$

fixed, and reduces the cost of evaluating the drift on a subset $\mathcal{Q}\subseteq\mathcal{K}_{\text{sde}}$:

$$\tilde{\mathbf{u}}_k = \begin{cases} \mathbf{u}_\theta(\mathbf{x}_{t_k}, t_k), & k\in\mathcal{Q},\\ \text{Predict}(\mathbf{x}_{t_k}, t_k, \mathcal{C}), & k\notin\mathcal{Q}, \end{cases} \tag{3}$$

where $\mathcal{C}$ is a decoder-reproducible cache. Equation 1 is then applied at every SDE step with the transmitted $\mathbf{z}_k^*$ unchanged. Multi-resolution coding follows the same principle spatially: early steps may run on a smaller latent grid, but the stage target must remain a projection of the final high-resolution latent so that the lifted trajectory is still a trajectory toward the same $\mathbf{x}_0$. The rest of this section instantiates these two ways of reducing generator cost while preserving the codebook correction channel.

### 3.3. Multi-resolution coding with a unified target

Resolution schedule. We replace the single-resolution trajectory with an $S$-stage progressive schedule, $s=0,\dots,S-1$, where stage $s$ operates on a latent of spatial size $(H_l^{(s)}, W_l^{(s)})$ with $H_l^{(0)}\le\cdots\le H_l^{(S-1)}=H_l$. Stage $s$ denoises for $\tau_s$ codebook steps; at transition step $\tau_{s\to s+1}$ the trajectory is lifted to the next resolution via a 2D DCT zero-pad in the new band:

$$\bar{\mathbf{x}}_{t_k}^{(s+1)} = \mathcal{T}^{-1}\Big(\text{Embed}\big(\mathcal{T}(\mathbf{x}_{t_k}^{(s)})\big) + t_k\,\boldsymbol{\epsilon}_H^{(s+1)}\Big), \tag{4}$$

where $\mathcal{T}$ is the spatial DCT, Embed pads the new high-frequency band with zeros, and $\boldsymbol{\epsilon}_H^{(s+1)}$ is a fresh Gaussian sample on that band. Equation 4 is spectrally correct: the low band passes through unchanged and the new band matches the marginal distribution expected by the generator at the new resolution and timestep $t_k$.

The target-domain mismatch. For the encoder to drive the low-stage trajectory, it needs a low-stage ground-truth latent $\mathbf{x}_0^{(s)}$. The natural choice is

$$\mathbf{x}_0^{(s),\,\text{naive}} \;=\; \text{VAE}^{(s)}\big(\text{down}(\mathbf{V})\big), \tag{5}$$

i.e., spatially downsample the pixel video and re-encode at the lower resolution. This is what a single-resolution codec would do at each scale independently. The decoder, however, does not produce $\mathbf{x}_0^{(s),\,\text{naive}}$ at the end of stage $s$: it produces the trajectory whose low band, after Eq. 4 lifts it, agrees with $\mathcal{T}_L(\text{VAE}(\mathbf{V}))$—the DCT-low projection of the high-resolution latent. The two are different objects:

$$\text{VAE}^{(s)}\big(\text{down}(\mathbf{V})\big) \;\neq\; \mathcal{T}_L\big(\text{VAE}(\mathbf{V})\big), \tag{6}$$

because the VAE encoder is nonlinear and its receptive field on the downsampled pixel support is not the

corresponding DCT projection of its receptive field on the full-resolution support. On UVG with the Wan 2.1 VAE we measure a relative gap $\|\text{lhs}-\text{rhs}\|/\|\text{rhs}\| \approx 0.41$ at $t = 0$ on the low band. The encoder under Eq. 5 drives the trajectory toward an object the decoder cannot produce; the high-band fill at transition cannot recover the mismatch.

Equivalently, the low stage is optimized against $\mathbf{x}_0^{(s),\,\text{naive}}$, but after spectral lift the full-stage loss is sensitive to

$$\mathcal{T}_L(\bar{\mathbf{x}}_{t_k}^{(s+1)}) - \mathcal{T}_L(\mathbf{x}_0), \tag{7}$$

not to $\mathbf{x}_{t_k}^{(s)} - \mathbf{x}_0^{(s),\,\text{naive}}$. Thus a perfect codebook match in the native low-resolution VAE domain can still become a biased initialization in the high-resolution codec domain.

Fix: unify both stages in the high-resolution latent domain. We replace Eq. 5 with

$$\mathbf{x}_0^{(s),\,\text{ours}} \;=\; \mathcal{T}_L\big(\text{VAE}(\mathbf{V})\big), \tag{8}$$

i.e., encode the pixel video once at full resolution and project to the low band by DCT truncation. Both stages now reference the same latent $\mathbf{x}_0 = \text{VAE}(\mathbf{V})$, observed at different spectral bandwidths. Transition (Eq. 4) stops being a domain change and becomes pure band expansion: stage $s$ converges to $\mathcal{T}_L(\mathbf{x}_0)$, and stage $s{+}1$ adds $\mathcal{T}_H(\mathbf{x}_0)$, recovering the same $\mathbf{x}_0$ that the encoder is matching against. The bitstream is unchanged; only the encoder's matching target moves to a spectrally consistent reference.

Transition codebook. At the transition step $\tau_{s\to s+1}$, the newly opened high band $\boldsymbol{x}_{t_k}^{H_{s+1}}$ is by default filled with pure noise $t_k\boldsymbol{\epsilon}_H$ (Eq. 4). The encoder knows the ideal high-band content $(1-t_k)\,\mathcal{T}_{H_{s+1}}(\mathbf{x}_0)$, so the trajectory-consistent residual is $\mathbf{r}_{H_{s+1}} = (1-t_k)\,\mathcal{T}_{H_{s+1}}(\mathbf{x}_0)$. We transmit one additional codebook atom per latent frame at this step, selected by inner product with $\mathbf{r}_{H_{s+1}}$. This costs $< 1\%$ of the per-GOP bit budget but recovers up to 0.4 dB PSNR over random high-band fill (Section 5). The transition high band therefore has the additive form

$$\mathbf{x}_{t_k}^{H} = t_k\boldsymbol{\epsilon}_H + \widehat{\mathbf{r}}_H, \qquad \widehat{\mathbf{r}}_H \approx (1-t_k)\mathcal{T}_H(\mathbf{x}_0), \tag{9}$$

which preserves the Gaussian noise floor while using transmitted bits only for the target-aware clean contribution.

### 3.4. Predictive skip via clean-endpoint caching

The dominant per-step cost is the video-generator evaluation $\mathbf{u}_\theta(\mathbf{x}_t, t)$. We aim to replace approximately half of these evaluations along the codebook-driven trajectory.

Why velocity caching fails. The naive choice is to cache the velocity itself:

$$\mathbf{u}_{k+1}^{\text{naive}} \;=\; \mathbf{u}_\theta(\mathbf{x}_{t_k}, t_k), \tag{10}$$

i.e., reuse the previous step's direction. This silently drifts: $\mathbf{u}_\theta$ is a function of both $\mathbf{x}$ and $t$, and the trajectory has moved away from $\mathbf{x}_{t_k}$ by both a deterministic drift and an encoder-injected codebook innovation in Eq. 1. The cached direction is computed at a stale state, not the current one.

The fix: cache the trajectory endpoint. Under the linear flow interpolant $\mathbf{x}_t = (1-t)\,\mathbf{x}_0 + t\,\boldsymbol{\epsilon}$, the generator's MMSE estimate of $\mathbf{x}_0$ at any point on the trajectory is

$$\hat{\mathbf{x}}_{0|t_k} \;=\; \mathbf{x}_{t_k} - t_k\,\mathbf{u}_\theta(\mathbf{x}_{t_k}, t_k). \tag{11}$$

$\hat{\mathbf{x}}_{0|t_k}$ is, geometrically, where the model thinks the trajectory ends. We cache this endpoint instead. When we skip the generator call at step $k{+}1$, we recover an approximate velocity from the cached endpoint and the current state and time:

$$\tilde{\mathbf{u}}_{k+1} \;=\; \big(\mathbf{x}_{t_{k+1}} - \hat{\mathbf{x}}_{0|t_k}\big)\,/\,t_{k+1}. \tag{12}$$

This is valid up to a local linearization around $\hat{\mathbf{x}}_{0|t_k}$: because the clean endpoint is invariant along the deterministic part of the trajectory under linear flow, only the codebook innovation between steps $k$ and $k+1$ moves the endpoint, and that movement is small relative to the cached value. Crucially, the full SDE/codebook correction in Eq. 1 is preserved at the skipped step; only the generator query is replaced.

Schedule and warmup. We adopt the fixed alternating schedule $k$ is run $\Leftrightarrow k \equiv k_0 \pmod 2$, where $k_0$ is the first SDE step in the current resolution stage. At every transition the cache is invalidated (the spatial resolution and therefore the operating domain changes), and we additionally delay the first post-transition skip by one step (transition warmup): the cache is rebuilt at step $\tau_{s\to s+1}{+}1$ before being reused at step $\tau_{s\to s+1}{+}2$. The number of generator calls is unchanged relative to the no-warmup schedule; the only difference is which step's output drives the first post-transition skip.

One-step radius. Empirically the cache is locally valid for one skipped step. We tested adaptive variants that allowed two or more consecutive skips when a

running per-step error estimate fell below a threshold; quality collapsed as soon as the cap of two was actually triggered (Section 5). The mechanism is recursive cache staleness: the second skipped step plugs an already-extrapolated state back into the same linearization, and the error compounds. This radius-of-one observation rules out a large family of more aggressive schedules and identifies the fixed alternating pattern as Pareto-optimal under this caching scheme.

### 3.5. GOP-level conditioning modes

We retain the three GOP-conditioning modes of GVCC [23] and apply both acceleration mechanisms uniformly across them.

T2V (intra-only). Each GOP is generated unconditionally from text. The per-GOP bitstream consists of codebook bits only and the operating point is the lowest in our set ($\leq$ 0.005 bpp at the bitrate hyperparameters of Section 4). This is the analogue of an all-I-frame codec.

I2V with autoregressive chaining. The first GOP takes the ground-truth first frame of the sequence as a "free" anchor; subsequent GOPs reuse the decoded last frame of the previous GOP as their anchor, avoiding repeated transmission of boundary frames. To suppress drift in the high-noise tail latent frames, we apply a small quantized residual correction (8-bit, last latent frame only) that the encoder transmits as a few hundred bytes per GOP.

FLF2V with boundary sharing. Each GOP is conditioned on both its first and last frames. For a sequence of $N$ GOPs, we transmit $N{+}1$ boundary frames (compressed with CompressAI [1] at quality 4): the first GOP transmits two frames; each subsequent GOP reuses the previous GOP's last frame as its own first and transmits only one new last frame. This boundary-sharing scheme amortizes the boundary cost over long sequences and provides dual temporal anchors that reduce per-frame variance within each GOP.

The three modes occupy distinct points on the rate-quality plane: Section 4 reports them jointly.

## 4. Experiments

### 4.1. Setup

Backbones. Unless otherwise noted, the video generator is Wan 2.1-T2V-1.3B for T2V and Wan 2.1-FLF2V-14B-720P for FLF2V, matching the backbones reported in [23]. I2V uses Wan 2.1-I2V-14B-720P. All checkpoints are used in bf16 without any fine-tuning or distillation.

Dataset. We evaluate on UVG-1080p [13] (seven sequences: Beauty, Bosphorus, HoneyBee, Jockey, ReadySteadyGo, ShakeNDry, YachtRide). Each sequence is split into 33-frame GOPs with no overlap; sequences are processed at their native $1920\times1080$ resolution and additionally at $1280\times720$ for cross-checks. The half-UVG evaluation in our main 1080p tables uses the first 9 GOPs ($\approx$ 300 frames) per sequence, 63 GOPs total; we extend to full UVG (18 GOPs per sequence) in the camera-ready version.

Hyperparameters. We follow the GVCC defaults: $T=20$ total sampling steps with $N=3$ deterministic ODE tail steps, codebook size $K=16384$, $M=64$ atoms per latent frame per step at 720p and $M=80$ at 1080p, SDE scale $g_{\text{scale}}=3.0$. The multi-resolution schedule uses $S=3$ stages at 1080p ($480\times832\rightarrow720\times1280\rightarrow1080\times1920$) and $S=2$ stages at 720p ($480\times832\rightarrow720\times1280$), with transitions at codebook steps 6 and 12 for $S=3$ and at step 8 for $S=2$. FLF2V uses CompressAI [1] quality 4 for boundary frames with $N{+}1$ boundary sharing.

Metrics. We report PSNR (mean across all decoded frames vs. ground truth), MS-SSIM, and LPIPS (AlexNet backbone). Bitrate is reported as bits-per-pixel (BPP) computed over $H\times W\times F$ pixels per GOP. Decoder wall-clock is the per-GOP time on a single NVIDIA RTX PRO 6000 Blackwell (96 GB) with bf16 inference and PyTorch 2.10, averaged across all GOPs of all sequences.

Compared methods. Traditional codecs HEVC [18] and VVC [2], learned codecs DCVC-FM [7] and DCVC-RT [5], generative codecs GLC-Video [15] and GNVC-VD [12], zero-shot generative codec Free-GVC [9], and the published GVCC [23] are reported on UVG-1080p with numbers taken from the respective papers. We mark our entries GVCCTurbo-{T2V, I2V, FLF2V}; each entry uses the full acceleration stack (multi-resolution coding with the unified target, predictive skip with $p=2$ alternating schedule and transition warmup, and the DLC1 transition codebook). The codec backbone is otherwise identical to the published GVCC of the same conditioning mode, so any change in PSNR/LPIPS is attributable to the acceleration mechanisms and the bitrate of the transition codebook.

Table 1. Comparison on UVG 1080p across three representative bitrate regimes. Format follows [23] (their Table 2): each GVCC variant is reported at its native operating point. Baseline LPIPS values are the published readings (AlexNet backbone for ours; baseline backbone may differ). Bold: lowest LPIPS per tier. The GVCCTurbo-T2V (1.3B) and GVCCTurbo-FLF2V rows are preliminary, half-UVG measurements (first 9 GOPs per sequence, 63 GOPs total); the GVCCTurbo-I2V row reuses the published GVCC-I2V configuration without re-evaluation. Full-UVG 1080p measurements are the next step.

| | Tier 1: ∼0.003 bpp | | | Tier 2: ∼0.006 bpp | | | Tier 3: ∼0.05 bpp | | |
|---|---|---|---|---|---|---|---|---|---|
| Method | PSNR↑ | LPIPS↓ | BPP | PSNR↑ | LPIPS↓ | BPP | PSNR↑ | LPIPS↓ | BPP |
| Traditional | | | | | | | | | |
| HEVC [18] | 29.8 | 0.385 | 0.003 | 30.2 | 0.360 | 0.006 | 34.7 | 0.115 | 0.050 |
| VVC [2] | 31.5 | 0.325 | 0.003 | 31.8 | 0.315 | 0.006 | 36.0 | 0.112 | 0.050 |
| Learned | | | | | | | | | |
| DCVC-FM [7] | 34.0 | 0.286 | 0.003 | 34.7 | 0.265 | 0.006 | 37.0 | 0.110 | 0.050 |
| DCVC-RT [5] | 34.1 | 0.285 | 0.003 | 34.6 | 0.261 | 0.006 | 39.7 | 0.115 | 0.050 |
| Generative (trained) | | | | | | | | | |
| GLC-Video [15] | 27.5 | 0.160 | 0.003 | 29.5 | 0.145 | 0.006 | 32.0 | 0.109 | 0.050 |
| GNVC-VD [12] | 26.8 | 0.165 | 0.003 | 30.0 | 0.140 | 0.006 | — | — | — |
| Generative (zero-shot) | | | | | | | | | |
| Free-GVC [9] | — | 0.185 | 0.003 | — | 0.155 | 0.006 | — | 0.105 | 0.050 |
| GVCC-T2V [23] | 29.7 | 0.133 | 0.0027 | | | | | | |
| GVCC-FLF2V [23] | | | | 31.7 | 0.117 | 0.006 | | | |
| GVCC-I2V [23] | | | | | | | 32.2 | 0.087 | 0.050 |
| GVCCTurbo-T2V† (1.3B) | 27.4 | 0.214 | 0.0030 | | | | | | |
| GVCCTurbo-FLF2V† (14B) | | | | 32.2 | 0.101 | 0.0070 | | | |
| GVCCTurbo-I2V‡ (14B, inherited) | | | | | | | 32.2 | 0.087 | 0.050 |

† Preliminary: first 9 GOPs of each UVG sequence (63 GOPs total). ‡ Reuses published GVCC-I2V configuration; not re-evaluated in this work.

### 4.2. Main results: UVG 1080p

Table 1 summarizes our main RD-quality comparison.

FLF2V (Tier 2, preliminary). GVCCTurbo-FLF2V at 1080p obtains 32.2 dB PSNR and 0.101 LPIPS at 0.0070 BPP on the first 9 GOPs of each sequence; we have not yet evaluated the full 18 GOPs per sequence. Against the published GVCC-FLF2V operating point, this improves PSNR by +0.5 dB and LPIPS by −14% at a 17% higher bitrate. The bitrate gap arises from the half-UVG boundary amortization: the $N$+1 boundary sharing scheme of Section 3.5 amortizes boundary cost over 9 GOPs in our run versus 18 in the published comparison. We project the gap to narrow by approximately 5% when extended to full UVG, but the projected number is not measured. Against all non-GVCC entries in Tier 2, our LPIPS is lowest by a clear margin (closest competitor: GNVC-VD at 0.140).

T2V (Tier 1, preliminary). GVCCTurbo-T2V at 1080p (Wan-T2V-1.3B backbone, half UVG) reaches 27.4 dB at 0.0030 BPP. This is 2.3 dB below the published GVCC-T2V operating point (Wan-T2V-14B at 29.7 dB) and reflects a backbone gap: the 1.3B model is significantly more efficient to decode but loses spatial fidelity at 1080p relative to the 14B model. The backbone-scaling axis is orthogonal to the acceleration mechanisms and was not optimized in this work. The matching T2V evaluation at 720p on full UVG is our main timing result and is discussed in Section 4.3.

I2V (Tier 3, inherited). We did not re-evaluate the I2V mode in this work. The GVCCTurbo-I2V row reuses the published GVCC-I2V configuration unchanged; our acceleration mechanisms are compatible with that configuration (Section 3.5) but the bitrate-quality operating point is dominated by the AR-chain tail-residual correction, which is unaffected by either of our two contributions. We expect decoder wall-clock to drop comparably to the T2V/FLF2V cases when the acceleration stack is applied, and we leave this confirmation to future work.

### 4.3. Decoder wall-clock

We report decoder wall-clock in two separate tables. The 720p T2V table (Table 2) is our main full-UVG speed measurement and isolates the cumulative effect of each acceleration component. The

Table 2. 720p T2V, full UVG. Cumulative effect of each acceleration component on Wan 2.1-T2V-1.3B. Vanilla is the GVCC pipeline without any of our mechanisms; +vec adds vectorized codebook RNG (an implementation fix); +vec+x0c adds predictive skip; the bottom row adds multi-resolution coding $S = 2$ (480 → 720). Time is per-GOP decoder wall-clock, averaged across 63 GOPs (7 seq × 9 GOPs in the published GVCC main table; full UVG with the same backbone in our equivalent main run).

| Method (T2V-1.3B at 720p) | Time/GOP | Speedup |
|---|---|---|
| Vanilla | ∼130s | 1.0× |
| +vec | 38s | 3.4× |
| +vec +x0c | 25s | 5.2× |
| +vec +x0c +MR ($S=2$) | 19s | 6.8× |

Table 3. 1080p FLF2V, half UVG (preliminary). The multi-resolution stack at 1080p uses $S = 3$ (480 → 720 → 1080), which spends a substantial fraction of its compute at the 1080p stage itself. Adding $S = 3$ MR to the 720p-friendly acceleration stack therefore yields a smaller speedup than at 720p; we report the trade-off rather than a single headline number.

| Method (FLF2V-14B at 1080p) | Time/GOP* | Speedup |
|---|---|---|
| Vanilla (estimated)‡ | ∼700s | 1.0× |
| +vec (estimated)‡ | ∼425s | 1.6× |
| +vec +x0c (single-resolution, est)‡ | ∼370s | 1.9× |
| +vec +x0c +MR ($S=3$, measured) | 439s | 1.6× |

*Half UVG (first 9 GOPs per sequence). ‡Estimated from the corresponding 720p measurement scaled by the latent-token count ratio; full-UVG single-resolution 1080p measurements are pending.

1080p FLF2V table (Table 3) reports the cost-quality trade-off of adding multi-resolution coding to a high-quality FLF2V configuration on the preliminary half-UVG subset. All measurements use a single NVIDIA RTX PRO 6000 (96 GB, bf16) and PyTorch 2.10.

720p T2V is the main speed story. Table 2 shows the 720p T2V decoder under the full acceleration stack at 19 s per GOP, a 6.8× speedup over the GVCC baseline. The decoder reconstructs 33 frames at 16 fps (≈ 2.06 s of video) in 19 s, i.e. ∼ 9× slower than playback. This is the closest we get to real-time-class decoding within the constraints of trajectory preservation; further reduction without changing the codec channel would require a distilled few-step generator.

1080p FLF2V is a quality-first operating point. At 1080p (Table 3), the $S = 3$ multi-resolution schedule allocates roughly a third of its codebook steps to the 1080p stage; these steps remain expensive even with predictive skip. The 439 s per GOP figure is therefore a deliberate trade-off: we exchange some of the 720p speed advantage for the +0.5 dB PSNR and −14% LPIPS reported in Table 1. The MR row remains ≈ 1.6× faster than the estimated single-resolution vanilla; the bulk of the residual cost is the 1080p stage itself.

Table 4. Multi-resolution coding with naive vs. unified low-stage target. Full UVG (7 seq × 3 GOPs) at 720p, Wan 2.1-T2V-1.3B, $S=2$ schedule (480→720). Δ is relative to the single-resolution baseline of 28.72 dB at 0.00483 BPP.

| Target | PSNR (dB) | BPP | ΔPSNR |
|---|---|---|---|
| Single-resolution baseline | 28.72 | 0.00483 | — |
| MR naive + random fill | 27.67 | 0.00453 | −1.05 |
| MR naive + DLC1 transition | 28.07 | 0.00481 | −0.65 |
| MR unified + DLC1 (ours) | 28.50 | 0.00481 | −0.22 |

## 5. Ablations

We isolate the contribution of each acceleration mechanism on full UVG 720p with the Wan 2.1-T2V-1.3B backbone, $T = 20$, $M = 64$, $K = 16384$, $g_{\text{scale}} = 3.0$ unless otherwise noted. Decode time is reported per GOP.

### 5.1. The target-domain mismatch and its fix

Table 4 reports our main multi-resolution ablation on full UVG (7 sequences × 3 GOPs) at 720p with Wan 2.1-T2V-1.3B. All rows share the same codebook hyperparameters ($M = 64$, $K = 16384$, $T_{\text{sde}} = 17$) and the same single-resolution baseline.

Is the residual gap a codebook or a target problem? The naive scheme with random high-band fill loses 1.05 dB. The DLC1 transition codebook recovers ≈ 0.4 dB at < 1% bitrate overhead, but a residual 0.65 dB gap remains. To check whether this gap is a codebook-expressiveness problem or a codec-target problem, we ran a diagnostic joint-oracle study on a two-sequence subset (Bosphorus + HoneyBee, 3 GOPs each), in which the encoder is allowed to inject the ground-truth high-band content at the transition step (non-deployable, zero bits transmitted for the oracle row). Table 5 reports the result. The numbers in Table 5 are not directly comparable to those of Table 4: the diagnostic subset's single-resolution baseline is 30.15 dB rather than 28.72 dB.

Under the naive target, even the joint oracle is capped at −1.26 dB on the diagnostic subset: the gap

Table 5. Joint-oracle diagnostic on a small subset (Bosphorus + HoneyBee, 3 GOPs each), Wan 2.1-T2V-1.3B at 720p, $S = 2$. The subset's single-resolution baseline is 30.15 dB. Numbers are not directly comparable to Table 4.

| Target | PSNR (dB) | Δ vs 30.15 |
|---|---|---|
| Single-resolution (subset baseline) | 30.15 | — |
| MR naive + random | 28.46 | −1.69 |
| MR naive + DLC1 | 28.72 | −1.43 |
| MR naive + joint oracle | 28.89 | −1.26 |
| MR unified + DLC1 (ours) | 30.05 | −0.10 |

Table 6. Caching choice for the per-step generator query, $p=2$ alternating schedule, full UVG 720p, multi-resolution $S = 2$. Time is per-GOP decoder wall-clock; ΔPSNR is relative to no caching.

| Cache | PSNR (dB) | Time/GOP | ΔPSNR |
|---|---|---|---|
| None (full $T$ generator calls) | 28.50 | 25s | — |
| Velocity cache (Eq. 10) | 26.71 | 19s | −1.79 |
| Clean-endpoint cache (ours, Eq. 12) | 28.45 | 19s | −0.05 |

is intrinsic to the codec target the encoder is matching against, not to codebook expressiveness. Under the unified target, the same diagnostic gap collapses to −0.10 dB. Both observations are consistent with the full-UVG numbers in Table 4.

Magnitude alignment is not the fix. We tried two natural variants of the unified target that rescale $\mathcal{T}_L(\mathrm{VAE}(\mathbf{V}))$ to match the per-channel std (or norm) of the naive target before use, intending to "soften" the domain gap. Both hurt: by ≈ 0.5 dB on the diagnostic subset relative to the raw unified target. Codebook-matching is spectrally sensitive, and rescaling re-introduces the domain mismatch in a more subtle form.

The fix is essentially one line of code. Replacing Eq. 5 with Eq. 8 collapses the full-UVG gap from −0.65 (naive + DLC1) to −0.22 dB at the same bitrate. The diagnostic-subset joint-oracle ceiling under the unified target is −0.10 dB, showing that once the target is corrected, transition high-band coding is no longer the dominant bottleneck.

### 5.2. Predictive skip: caching choice and schedule

Table 6 compares no caching, velocity caching, and clean-endpoint caching on UVG-720p with Wan 2.1-T2V-1.3B, at the fixed alternating $p=2$ schedule (every other SDE step skipped) with transition warmup after the multi-resolution lift.

Table 7. Adaptive predictive skip with a per-stage cap on consecutive skips. HoneyBee, single GOP, 720p, Wan-T2V-1.3B. Threshold tuned to keep average skip rate near 50%.

| Schedule | PSNR (dB) | Time/GOP |
|---|---|---|
| Fixed alternating ($p=2$, cap ≡ 1) | 29.47 | 18s |
| Adaptive, cap =1, thr 0.30 | 29.48 | 19s |
| Adaptive, cap =2, thr 0.15 (triggered) | 29.08 | 17s |
| Adaptive, no cap, thr 0.40 (triggered) | 25.95 | 13s |

The clean-endpoint cache reduces decoder time by 1.32× at a 0.05 dB PSNR cost; the same skip rate with naive velocity caching costs 1.79 dB—a 36× larger PSNR drop. The two schemes are otherwise identical (same skip schedule, same number of generator calls); the difference comes entirely from what is cached. The velocity cache is computed at a stale state and the trajectory has moved away from $\mathbf{x}_{t_k}$ by both the deterministic drift and the codebook innovation between steps $k$ and $k+1$. The endpoint cache is geometrically invariant along the deterministic part of the trajectory, so only the codebook-induced shift between steps remains as approximation error.

The one-step radius. Table 7 ablates the maximum allowed number of consecutive skips, holding the average skip rate fixed via an adaptive threshold (we transmit a 1-bit-per-SDE-step skip mask in the bitstream when adaptivity is on).

Whenever the cap of 2 is actually triggered (i.e., the adaptive controller picks two consecutive skips), quality drops sharply. Removing the cap entirely allows long skip runs and quality collapses by 3.5 dB. With a hard cap of 1, the adaptive controller can only cancel a would-be-skip when the running per-step error is too large, never schedule a faster pattern; quality therefore matches the fixed alternating schedule but speed is bounded above by the same. Adaptive caching with cap ≡ 1 is a robustness mechanism, not a new acceleration frontier. The fixed alternating schedule is Pareto-optimal under the clean-endpoint caching scheme.

### 5.3. Stage count and transition placement (diagnostic subset)

Table 8 ablates the number of multi-resolution stages $S$ and the placement of transition steps. The schedule sweep is relatively expensive (six configurations × multiple sequences), so we run it on a diagnostic subset—Bosphorus and HoneyBee, 3 GOPs each—rather than on full UVG. The single- resolution baseline for this subset is 30.15 dB, distinct from the 28.72 dB full-UVG baseline used in Table 4; the two tables answer

Table 8. Diagnostic-subset stage-count and transition-placement sweep (Bosphorus + HoneyBee, 3 GOPs each, Wan-T2V-1.3B at 720p). $\tau$ lists the SDE step indices at which each transition occurs. $\Delta$PSNR is relative to this subset's single-resolution baseline of 30.15 dB (not the full-UVG 28.72 dB baseline of Table 4).

| Schedule | $S$ | $\tau$ | PSNR (dB) | BPP |
|---|---|---|---|---|
| Single resolution | 1 | — | 30.15 | 0.00483 |
| 480→720 | 2 | {8} | 28.72 | 0.00481 |
| 480→720 (early $\tau$) | 2 | {6} | 28.51 | 0.00481 |
| 480→720 (late $\tau$) | 2 | {10} | 28.65 | 0.00481 |
| 272→480→720 | 3 | {4, 12} | 27.76 | 0.00506 |
| 272→480→720 (oracle) | 3 | {4, 12} | 28.07 | 0.00451 |

different questions on different data and their numbers should not be combined. All entries below use the unified target (Eq. 8) and the DLC1 transition codebook.

$S=2$ is the sweet spot on Wan 2.1. Two stages with a transition at the midpoint dominate $S=3$ schedules that include a 272p floor. The drop at $S=3$ is not a coding artifact: even the joint oracle at $S=3$ (28.07 dB) cannot reach the $S=2$ codebook result (28.72 dB). The cause is that Wan 2.1 was not trained at latents below the 480p band; running the first few denoising steps on an even smaller latent operates the generator out-of-distribution, and the spectrally lifted state at transition is no longer a good initialization for the higher-resolution stage. Moving the low floor up to 480p eliminates the problem; further increasing $S$ within $[480, 720]$ adds transition overhead without quality benefit.

Transition timing. At $S=2$, sweeping the transition step in $\{6, 8, 10\}$ yields 28.51/28.72/28.65 dB respectively. The step-8 choice (approximately midway through the SDE phase) is best by a small margin (+0.21 over step-6). This matches the spectral diffusion intuition [21]: too early a switch deprives the low stage of useful denoising, too late a switch wastes generator capacity at high resolution.

### 5.4. Vectorized codebook RNG: implementation, not algorithm

The Turbo-DDCM [19] codebook construction generates $K$ atoms per step per latent frame by independent randn $(D)$ calls. In our implementation, replacing these with a single batched randn $(K, D)$ per step reduces decoder wall-clock by $\approx 1.9\times$. The batched call produces a different per-atom RNG sequence than the sequential call, so the two settings emit different codebook indices for the same input; what they preserve is the bit budget and the deterministic encoder/decoder reproducibility (both sides agree as long as both use the same batching). This is an engineering fix—it does not appear in the bitstream—but we report it for transparency: all timing numbers in this paper, including the published GVCC baseline rows in Table ??, include this fix unless explicitly marked vanilla.

## 6. Conclusion

We have presented GVCCTurbo, a zero-shot generative video codec that retains the codebook-driven backbone of GVCC while reducing the cost of its expensive video-generator trajectory. The organizing principle is trajectory preservation: keep the SDE/codebook correction grid that carries the compressed residual, but avoid unnecessary generator work where the missing drift can be reconstructed reproducibly. Two mechanisms instantiate this principle—unifying multi-resolution stages in a single latent domain, and caching the trajectory endpoint rather than the velocity for predictive skipping. Both are simple in implementation, require no retraining, and leave the bitstream format unchanged beyond the optional DLC1 transition codebook.

The experiments support two broader lessons. First, multi-resolution coding is viable for generative compression only when all stages share the same latent target; otherwise the low-resolution stage optimizes a different object from the full-resolution decoder. Second, model-call skipping should cache the clean endpoint, not the velocity: the endpoint remains a stable destination, whereas the velocity is tied to a stale state. These lessons are codec-specific and do not follow from ordinary fast-sampling recipes.

Limitations and future work. The decoder is still substantially slower than learned codecs of the DCVC family ($\sim 10^2$ vs. $\sim 10^{-1}$ s/GOP), and the main remaining cost is the per-generator-call latency of the high-resolution stage. Two directions appear most promising. First, the predictive-skip mechanism's one-step-radius observation rules out simple multi-step caching, but does not preclude a learned step-distillation that retrains a few-step generator compatible with the encoder's per-step reproducibility requirement—this is an open research direction with no clean fix today. Second, a lightweight generator natively trained at 1080p (the closest existing candidate at the time of writing, LTX-Video [8], is a strong starting point) would change the backbone-vs-codec balance of the cost; we plan to investigate this trade in future work.

We release code, full per-GOP measurements, and codebook bitstreams to support reproducibility and

further investigation along the multi- resolution and predictive-skip axes.

## References

[1] Jean Bégaint, Fabien Racapé, Simon Feltman, and Akshay Pushparaja. CompressAI: a PyTorch library and evaluation platform for end-to-end compression research. In arXiv preprint arXiv:2011.03029, 2020. 6

[2] Benjamin Bross, Ye-Kui Wang, Yan Ye, Shan Liu, Jianle Chen, Gary J. Sullivan, and Jens-Rainer Ohm. Overview of the versatile video coding (VVC) standard and its applications. IEEE Transactions on Circuits and Systems for Video Technology, 31(10):3736–3764, 2021. 3, 6, 7

[3] Patrick Esser et al. Scaling rectified flow transformers for high-resolution image synthesis. In International Conference on Machine Learning (ICML), 2024. 3

[4] Jonathan Ho, Chitwan Saharia, William Chan, David J. Fleet, Mohammad Norouzi, and Tim Salimans. Cascaded diffusion models for high fidelity image generation. In Journal of Machine Learning Research, 2022. 3

[5] Zhihao Jia, Bin Li, Houqiang Li, and Yan Lu. DCVC-RT: Real-time neural video compression with feature modulation. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025. 1, 3, 6, 7

[6] J. Kim et al. Inference-time acceleration of diffusion models via velocity caching. arXiv preprint, 2025. 3

[7] Jiahao Li, Bin Li, and Yan Lu. Neural video compression with feature modulation. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. DCVC-FM. 1, 3, 6, 7

[8] Lightricks. LTX-Video: Realtime video latent diffusion, 2025. arXiv:2501.00103. 10

[9] Yi Ling et al. Free-GVC: Training-free generative video compression via reverse channel coding on pretrained diffusion models. arXiv preprint, 2026. 3, 6, 7

[10] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. In arXiv preprint, 2023. 3

[11] Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating diffusion models for free. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. 3

[12] Hao Mao et al. GNVC-VD: Generative neural video compression with visual diffusion. arXiv preprint, 2025. 3, 6, 7

[13] Alexandre Mercat, Marko Viitanen, and Jarno Vanne. UVG dataset: 50/120fps 4K sequences for video codec analysis and development. In ACM Multimedia Systems Conference, 2020. 6

[14] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL: Improving latent diffusion models for high-resolution image synthesis. In International Conference on Learning Representations (ICLR), 2024. 3

[15] Yifan Qi et al. GLC-Video: Generative latent compression for video at ultra-low bitrates. arXiv preprint, 2025. 3, 6, 7

[16] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In International Conference on Learning Representations (ICLR), 2022. 3

[17] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In International Conference on Machine Learning (ICML), 2023. 3

[18] Gary J. Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. Overview of the high efficiency video coding (HEVC) standard. IEEE Transactions on Circuits and Systems for Video Technology, 22(12): 1649–1668, 2012. 3, 6, 7

[19] Yair Vaisman et al. Turbo-DDCM: Fast and accurate single-image compression with multi-atom codebooks. In Advances in Neural Information Processing Systems, 2025. 3, 4, 10

[20] Wan-AI Team. Wan: Open and advanced large-scale video generative models, 2025. Wan 2.1, Alibaba. 2, 3

[21] Tianbo Xiao et al. Spectral progressive diffusion. In arXiv preprint arXiv:2605.18736, 2026. 2, 3, 10

[22] Enze Xie, Junsong Chen, Junyu Chen, Han Cai, et al. SANA: Efficient high-resolution image synthesis with linear diffusion transformer. In arXiv preprint, 2024. 3

[23] Ziyue Zeng, Xun Su, Haoyuan Liu, Bingyu Lu, Yui Tatsumi, and Hiroshi Watanabe. GVCC: Zero-shot video compression via codebook-driven stochastic rectified flow. arXiv preprint arXiv:2603.26571, 2026. 1, 3, 4, 6, 7